# Targeting Alignment: Extracting Safety Classifiers of Aligned LLMs

Jean-Charles Noirot Ferrand
*University of Wisconsin-Madison*
*jcnf@cs.wisc.edu*

## Abstract

Alignment in large language models (LLMs) is used to enforce guidelines such as safety. Yet, alignment fails in the face of *jailbreak* attacks that modify inputs to induce unsafe outputs. In this paper, we present and evaluate a method to assess the robustness of LLM alignment. We observe that alignment embeds a safety classifier in the target model that is responsible for deciding between refusal and compliance. We seek to extract an approximation of this classifier, called a surrogate classifier, from the LLM. We develop an algorithm for identifying candidate classifiers from subsets of the LLM model. We evaluate the degree to which the candidate classifiers approximate the model's embedded classifier in benign ($F_1$ score) and adversarial (using surrogates in a white-box attack) settings. Our evaluation shows that the best candidates achieve accurate agreement (an $F_1$ score above 80%) using as little as 20% of the model architecture. Further, we find attacks mounted on the surrogate models can be transferred with high accuracy. For example, a surrogate using only 50% of the Llama 2 model achieved an attack success rate (ASR) of 70%–a substantial improvement over attacking the LLM directly, where we only observed a 22% ASR. These results show that extracting surrogate classifiers is a viable (and highly effective) means for modeling (and therein addressing) the vulnerability of aligned models to jailbreaking attacks.

## 1 Introduction

Improvements in machine learning–specifically the introduction of the transformer architectures [38]–have led to increasingly powerful large language models (LLMs). Whether they are accessed through an API or interface (e.g.,GPT [26], Gemini [33], Claude [1]) or published as open-source (e.g.,Llama 2 [23], Llama 3 [24], Qwen [37], etc.), these models have become the de facto tool for tasks involving natural language. They serve as foundations for new tools, in which they are augmented with new capabilities [2, 25] or fine-tuned on a downstream task [11]. In adapting these models to tasks, LLMs undergo an alignment process in which they are further trained (refined) to satisfy safety objectives [23, 30]. For example, alignment guidelines often prevent a model from outputting responses that would be offensive, discriminatory, or harmful (i.e., unsafe). It is acknowledged that alignment can (and often does) fail in adversarial settings, resulting in a *jailbreak*: an unsafe input being accepted as safe. Indeed, many studies have explored the automated generation of jailbreak inputs, i.e., adversarial examples [18, 20, 44, 55].

We posit a novel approach to reason about LLM alignment in the context of jailbreak attacks. In this paper, we hypothesize that alignment embeds a "safety" classifier into a target model that determines whether an input prompt is safe (compliant with alignment goals) or unsafe (refusal with alignment). Consistent with this hypothesis, previous work has shown that the representations of safety queries are well separated in aligned models [19, 53], specifically in the first decoders within the LLM [16]. This work seeks to extract an approximation of the model classifier (called a *surrogate classifier*) and evaluate its utility in assessing a model's robustness to adversarial inputs. In this, we explore: (a) how a surrogate classifier can be accurately extracted, and (b) how the surrogate classifier can be used to assess the safety of or attack the aligned model.

We begin by exploring methods for extracting surrogate classifiers. Here, our approach systematically identifies candidates from a subset of the model architecture. Candidates are built by (a) finding a structure, i.e., a subset of the architecture of the model, that best represents the embedded safety classifier, (b) adding a classification head, and (c) training this classification head on unsafe and safe inputs to map the features extracted from the structure to the model predictions (positive for refusal, negative for compliance). The candidates are then evaluated in benign and adversarial settings. For the benign settings, we measure how well the candidate classifier agrees with the model's safety classifier on safe and unsafe inputs. For the adversarial settings, we measure transferability of adversarial inputs between the target model and candidate

classifiers to determine how well the latter capture the decision of the embedded safety classifier.

We perform a detailed analysis of the efficacy of our approach over 5 models and 2 datasets. We first evaluate candidates with respect to benign inputs, assessing whether each candidate classifier agrees with the safety classification of the LLM model. We find that the best candidates achieve accurate agreement (an $F_1$ score above 80%) using as little as 20% of the model architecture. Next, we evaluate the candidate classifiers with respect to transferability of adversarial inputs (i.e., as generated using best-in-class jailbreaking algorithms) from the candidates onto the LLM. These latter experiments show that some candidate classifiers are extremely effective tools for targeting LLMs. For example, a surrogate classifier using only 50% of the Llama 2 model achieved an attack success rate (ASR) of 70%–a substantial improvement over attacking the LLM directly, where we observed a 22% ASR.

Our results suggest that investigating the robustness can be furthered (and in many cases made more efficient) by extracting surrogate classifiers that target alignment tasks. In this paper, we have found that approximations of LLMs may be sufficient to explore the security of their deployment–and at the same time offer potential vectors for developing more effective attacks on alignment.

Our contributions are as follows:

- We explore the existence and structure of safety classifiers in LLMs. We show how these classifiers are used to decide whether an input complies with the alignment.
- We develop algorithms to extract highly accurate surrogate safety classifiers from LLMs by selectively extracting and refining subsets of LLM architectures.
- We evaluate the candidate classifiers with respect to their size and the ability to transfer attacks onto the victim LLM. We show that transferred attacks are more successful than direct attacks on the model in several settings.

# 2 Background

## 2.1 Large Language Models (LLMs)

**Large language model.** A LLM models the conditional probability of the next token (a unit of text for the model) with respect to previous tokens. Let $V$ be the vocabulary of the tokens and $V^* = \bigsqcup_{n=1}^{N} V^n$ the corresponding input space of sequences, where $N$ denotes the context window of the model (maximum amount of tokens that can be processed by the model). Given a sequence of tokens $x = x_1 x_2 \ldots x_T \in V^*$, the model aims to produce the next token $x_{T+1}$ by approximating the probability distribution.

$$p(x_{T+1} | x_1 x_2 \ldots x_T) \tag{1}$$

LLMs learn a vector representation of words called the embedding space. Given a sequence of tokens $x = x_1 x_2 \ldots x_T$, each token is assigned to an embedding, resulting in a sequence of vectors $\{h_t\}_{t \in \{1,\ldots,T\}}$ all in $\mathbb{R}^d$, where $d$ is the dimension of the embedding space. This study focuses on aligned chat models, which are almost exclusively of the same architecture: a sequence of *decoders*. The $i$-th decoder transforms a sequence of embeddings $h^{(i)}$ into another $h^{(i+1)}$, keeping the same sequence length. If the model is made of $D$ decoders, the probability distribution on the next token is obtained by applying a linear layer followed by a softmax function on the last output embedding of the last decoder, $h_T^{(D)}$, i.e., $p(\cdot | x_1 x_2 \ldots x_T) = \text{softmax}(A h_T^{(D)})$, where $A \in \mathbb{R}^{d \times |V|}$ is the learned matrix that maps the embeddings to the scores (or logits) for each token.

**Chat template.** By construction, LLMs simply predict the next token. Thus, before deployment, the model undergoes *instruction tuning*: a process which encodes "roles" in the model such that it can interact with a user. Part of this process includes a *chat template*: a fixed modification of the input to dissociate the roles. It relies on special tokens from the model to enforce a format. As a result of different training settings, different models generally come with different chat templates. Formally, for a given model, there exists a template $(x_{\text{pre}}, x_{\text{post}}) \in V^* \times V^*$ such that any input text $x$ is preprocessed by prepending $x_{\text{pre}}$ and appending $x_{\text{post}}$.

## 2.2 Alignment and Safety

**Alignment.** Since the training data used for LLMs comes from diverse sources, biases and undesirable behaviors emerge [9]. In order to prevent those behaviors from happening, models often go through an alignment process that regulates their outputs according to given guidelines. Several methods exist and are not mutually exclusive [23]. Approaches such as supervised fine-tuning (SFT) uses sample answers from humans, while reinforcement learning with human feedback (RLHF) [3, 6] trains a neural network that acts as a reward for the LLM. Since training another reward model may be costly, direct preference optimization (DPO) [30] uses the LLM as its own reward model.

**Guidelines and taxonomies.** The existence of alignment as a technique is a direct product of the complexity of identifying what is precisely expected from models (i.e., their objective). Safety is a broad term, thus, previous work has identified taxonomies [13, 41] that represent a broad range of unsafe behaviors. This allows researchers to evaluate alignment on a finer scale and understand where it could be improved. For example, when the prompts are decomposed into different categories such as self-harm, privacy, harassment, etc., models tend to refuse the prompts related to self-harm and accept the prompt related to privacy [8].

**Identifying unsafe outputs.** Similarly to their inputs, classifying LLMs outputs is challenging. Since aligned models tend to create strong refusal answers for detected unsafe prompts, a naive solution is to test whether certain refusal keywords are in the output (e.g.,*"Sorry"*, *"as a responsible AI"*, etc.) [55]. Alternatively, judge models (also LLMs) are trained to classify the safety of outputs with more [13] or less [22] granularity on the classes they predict, similarly to the taxonomies explained above. A limitation of judge models is that they can also be attacked [21, 31], leading to a possible high rate of false negatives (unsafe outputs judged as safe).

## 2.3 Attacks on LLMs

LLMs, like any class of machine learning models, are prone to attacks. While several exploits exist (e.g.,model-stealing [4], text misclassification, etc. [52]), the *jailbreak* exploit remains the most prevalent. Jailbreak refers to a compliance from an aligned LLM for an unsafe query, as defined by the alignment guidelines. Jailbreak attacks fall into one of two threat models: the white-box threat model, in which model parameters are known, and the black-box threat model, in which only access to the output of the model is assumed.

**White-box.** These attacks have been studied on open-weights models such has Llama 2 [23], Llama 3 [24], or Vicuna [5]. The most prevalent, GCG [55], uses greedy coordinate gradient (GCG) to mitigate the absence of mapping from embeddings (i.e., vectors) to tokens. Following this, many new white-box algorithms have been introduced [7, 42], improving existing algorithms [18, 19, 28, 46], using different objectives [48], or using specific properties of the model (e.g.,logits [17], special tokens [49], or generation [12]. In terms of efficacy, attacks like GCG have an average attack success rate (ASR) above 50% [22] on most open-source models.

**Black-box.** These attacks have been widely studied, given their prevalence in practical scenarios [1, 26]. Most of them are driven by empirical heuristics: they apply transformations to the initial prompt such as ASCII-art [14], translation into underrepresented languages [43], encoding and decoding [10], or using pre-existing templates [45]. Black-box attacks achieve remarkable efficacy. For instance, GPT-Fuzzer [44] can achieve ASR as high as 90% on commercial LLMs such has ChatGPT.

# 3 Methodology

This section outlines our approach for extracting a surrogate classifier from LLMs, which serves as the basis for our experiments. Before we detail our methodology, we first explain a preliminary experiment whose results inform our approach.

## 3.1 Preliminaries

As explained previously, LLMs are aligned but are prone to attacks. Regardless of the threat model, jailbreak attacks have given limited understanding of where alignment fails since they involve heuristics in either the adversarial objective or the perturbation they apply to the input. In this work, we hypothesize that alignment embeds a safety classifier in the model, whose existence we outline next. By extracting this safety classifier, we can create a new class of optimization-based attacks that reduces jailbreaking to misclassification on this surrogate classifier.

**Motivation of existence.** Refusal from aligned models often comes with a set of predefined outputs (e.g.,*"I cannot fulfill your request. I'm just an AI"*) as a product of alignment. As the space of refusal outputs is very limited, such outputs can be easily detected. We assume the following: the first generated token after the input is enough information to know whether the output will be a refusal or not. Under this assumption, there exists a set of refusal tokens $V_r$ such that, for a given input sequence of tokens $x = x_1 x_2 \ldots x_T \in V^*$, $x_{T+1} \in V_r$ if and only if the output answer is a refusal. Thus, classifying unsafe inputs reduces to verifying $x_{T+1} \in V_r$. In a deterministic generation (i.e., without sampling), this is done by finding the token $x_{T+1}$ that satisfies $x_{T+1} = \arg\max_{i \in V} \text{softmax}(Ah_T^{(D)})_i = \arg\max_{i \in V} [Ah_T^{(D)}]_i$. As explained in Section 2.1, the last embedding (after the last decoder) $h_T^{(D)}$ entirely determines the next token. Moreover, the sequence of decoders in the model transforms the sequence of embeddings such that for a given decoder $i$, the $t$-th element $h_t^{(i)}$ is a function of $h_1^{(i-1)}, \ldots, h_t^{(i-1)}$. Thus, the partial information on the classification contained in $h_1^{(i-1)}, \ldots, h_T^{(i-1)}$ propagates to $h_T^{(i)}$ and so forth until it reaches $h_T^{(D)}$.

**Separation metric.** For the classifier to exist, there must be separation between the two classes, i.e., inputs classified as unsafe (refusal) and inputs classified as safe (compliance). To demonstrate this, we measure the silhouette score on the embeddings associated to the two types of inputs (safe and unsafe). We use the silhouette score to determine how well clustered the two safety classes are and evaluate if there is indeed a separation, the metric is defined as follows:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \tag{2}$$

where $a(i)$ denotes the mean intra-cluster distance (average distance to all other points in the same cluster) and $b(i)$ the mean nearest-cluster distance (average distance to points in the nearest cluster).

This score ranges from $-1$ to 1 where scores below 0 mean that unsafe and safe representations overlap, and scores above 0.25 or 0.5 imply a weak or reasonable separation [32].

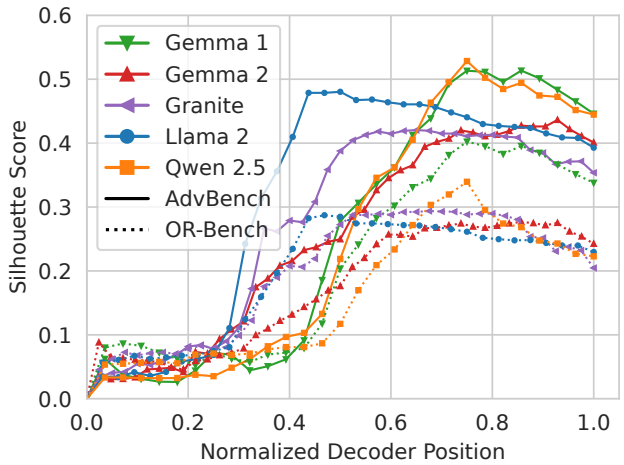



Figure 1: Silhouette score of the embeddings for different models on two datasets.

**Result.** We measure this score on five LLMs and two datasets (see Section 4.1 for the detailed experimental setup). Figure 1 shows the silhouette score of the embeddings for different models and datasets, with respect to the normalized decoder position (ratio between the current decoder position and the total number of decoders).

While it would be expected to see a nondecreasing trend, we see that after the silhouette score reaches its maximum, it starts to decrease. A possible explanation for this phenomenon is rooted in the fact that models are designed to perform multiple tasks. Therefore, after extracting and integrating the safety information from the input, the other tasks add more information to the embedding that is not related to safety. Thus, the safety separation decreases, but remains high enough for the classification. Besides the difference in degree of separation between the two datasets, we note that the trend is exactly the same.

**Takeaway 1**: Existence of a safety classifier

Across models and datasets, internal decoders show separation between safe and unsafe inputs that exceeds that of the final decoder, suggesting the existence of an embedded safety classifier that does not span the full model architecture.

## 3.2 Threat Model

Motivated by our preliminary results, our adversary aims to extract an approximation of the safety classifier of a target aligned model, called a surrogate classifier. A successful extraction would allow subsequent attacks to be made with more effective white-box jailbreak approaches, while forgoing the complex computational costs they incur. We will extract the surrogate classifier using subsets of the model, thus white-box access is assumed (i.e., the adversary has access to the model weights and architecture).

In jailbreak attacks, the adversarial objective is to make the model comply with an unsafe input by adding a perturbation. Aligned models overfit on the unsafe inputs. As a result, they can reject safe prompts [8], exhibiting a high sensitivity to certain tokens. While previous work has mostly studied jailbreaks (unsafe inputs identified as compliant), in this work we focus on extracting the surrogate classifier therein enabling attacks that target alignment more closely. To do so, we also study safe inputs identified as unsafe, as we hypothesize that they also provide substantial information on the decision boundary of the classifier.

## 3.3 Problem Formulation

**General formulation.** Let $f_\theta$ be a LLM with context window $N$ and vocabulary $V$, and let $\mathcal{R} : x \in V^* \mapsto C$ be a classification rule that maps the output of the LLM to a class from $C$. We aim to extract a *classifier* from the LLM through two components: a *structure* and a classification head. A *structure* $f_{\theta^*}, \theta^* \subset \theta$ is a subset of the model (e.g.,a decoder). A classification head $\mathcal{C}$ maps the representations learned by the structure to a class such that the classifier $\mathcal{C} \circ f_{\theta^*}$ is equivalent to the LLM for the classification task. Formally, for any token sequence $x \in V^*$, the following holds:

$$\mathcal{R}(f_\theta(x)) = \mathcal{C}(f_{\theta^*}(x)) \tag{3}$$

**Safety.** In this paper, we focus on safety classification, i.e., distinguishing between unsafe and safe inputs. This is a binary classification problem, where unsafe prompts are considered positive observations and safe prompts as negative observations. A jailbreak is then a false negative, while a false positive is "over-refusal" [8]. The classification rule $\mathcal{R}$ to assign a predicted LLM label is then comparable to how attack success is measured for jailbreak attacks, i.e., by verifying refusal or compliance. The classification rule $\mathcal{R}$ we chose is a model trained to classify LLM outputs as refusal or compliance as we focus on the type of answer from the LLM, not if it produces unsafe information.

## 3.4 Extracting the Classifier

In order to extract the classifier, we first need to have a set of candidates. We build a candidate classifier through the following three steps, summarized in Figure 2:

1. Select a subset $f_{\theta^*}$ of the architecture of the LLM $f_\theta$, which we will refer to as a *structure*.
2. Collect data points from the structure $(f_{\theta^*}(x), \mathcal{R}(f_\theta(x)))$ consisting in the features extracted from the structure and the predicted label of the LLM.

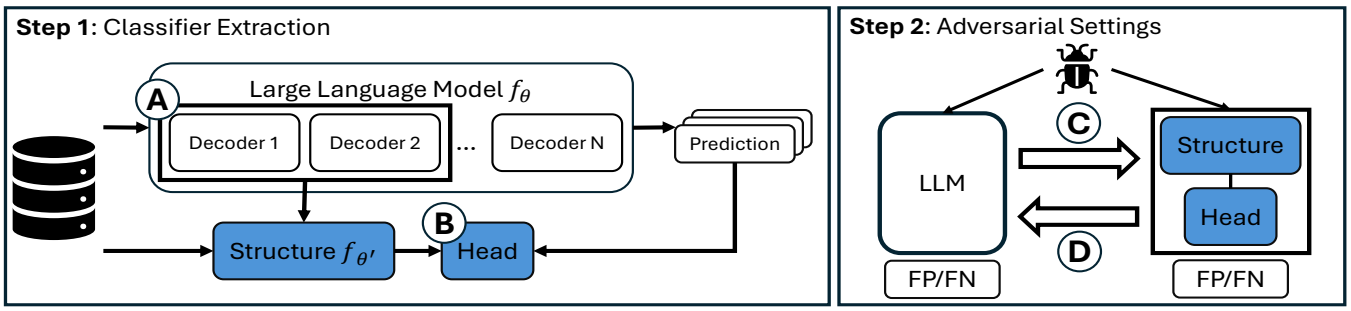



Figure 2: Methodology overview. In the first step, we extract the safety classifier of an LLM by (A) selecting a structure within the model and (B) training a classification head on the predictions of the LLM to create a candidate classifier. We verify its performance in benign setting. Then, we measure the transferability between the LLM and the candidate in both ways, i.e., adversarial examples of the LLM (C) transfer to the candidate classifiers and adversarial examples of the candidate classifiers (D) transfer to the LLM.

3. Train a classification head $\mathcal{C}$ on $(f_{\theta^*}(x), \mathcal{R}(f_\theta(x)))$.

The resulting classifier $\mathcal{C} \circ f_{\theta^*}$ will be termed a *candidate* classifier.

**Structures.** As explained in Section 2.1, most LLMs are built as sequences of decoders: processing units that take a sequence of vectors and output another sequence of vectors of the same length. LLMs can be scaled either in depth by adding decoders or in width by increasing the dimensionality of embeddings. Therefore, there are many ways to "cut" the model and thus many possible structures. It is clear that there is a separation between unsafe and safe representations at the end of aligned models [19] largely due to the strong refusal outputs. In addition, previous work [16, 50] has shown that separation occurs in early decoders [16]. Thus, we estimate the location of the safety classifier via two hypotheses. First, the classifier is at the decoder level. Second, the information relevant to the classification at a given decoder is contained at the end of the sequence (as discussed in Section 3.1).

We will refer to $f_{i,i+\delta}$ as the set of decoders between the $i$-th and the $i+\delta$-th decoder, $\delta \geq 0$. The second hypothesis implies that for a given input $x \in V^*$ where $f_{i,i+\delta}(x) \in \mathbb{R}^{|x| \times d}$, we only consider the last element of the sequence, reducing the input dimension of the classification head $\mathcal{C}$ to $\mathbb{R}^d$.

By construction, the input space of $f_{i,i+\delta}$ depends on the output space of $f_{1,i-1}$, since the model was trained with the entire architecture. In this work, we set $i$ to 1. We leave the exploration of structures that can isolate intermediate decoders to future work, as they would require techniques to mitigate the absence of earlier decoders, which is out of the scope of this work.

Finally, $\delta$ will be termed the *candidate size* and we will refer to $\frac{\delta}{D}$ as the *normalized candidate size*, i.e., the proportion of decoders of the LLM used by the candidate.

**Training.** For the selected structure $f_{i,i+\delta}$, the goal is to train a classification head $\mathcal{C}$ to obtain a candidate classifier. The first step is to create a dataset $\mathcal{X} = (f_{i,i+\delta}(x), \mathcal{R}(f_\theta(x)))$ consisting in the extracted feature from the structure $f_{i,i+\delta}$ and the predicted label[1] by applying the classification rule $\mathcal{R}$ to the output of the LLM $f_\theta$ for each prompt $x$.

Then, the classification head is trained following this objective:

$$\min_{(h,y)\in\mathcal{X}} \mathcal{L}(\mathcal{C}(h), y) \tag{4}$$

This results in a candidate classifier $\mathcal{C} \circ f_{i,i+\delta}$ of size $\delta$.

## 4 Evaluation

We apply our approach to answer the following research questions:

**RQ1.** How accurate are the candidate classifiers at approximating the safety classifier in benign settings? (Section 4.2)

**RQ2.** Does the performance of candidate classifiers hold in adversarial settings? (Section 4.3)

**RQ3.** Can we jailbreak aligned LLMs by attacking the surrogate classifier? (Section 4.4)

### 4.1 Experimental Setup

The dataset creation and attack experiments were run on NVIDIA A100 GPUs with 40 GB of VRAM. Attack experiments on the models amount to an approximate total of 3000 GPU hours. Attack experiments on the candidate classifiers amount to an approximate total of 500 GPU hours.

**Models.** We evaluate our approach on five open-weights aligned chat models, all from Hugging Face `Transformers 4.46.2` [40] library: `Llama-2-7b-chat` [23], `Qwen2.5-7B-Instruct` [37]

[1] Since we focus on extracting the classifier and not improve it, it is beneficial to have both right and wrong labels from the classifier

gemma-7b-it [35], gemma-2-9b-it [34], and granite-3.1-8b-instruct [36]. We chose these models because they achieve good performance at the classification task on the datasets considered, translating to a stronger alignment.

**Dataset augmentation.** Since most of the datasets lack safe prompts or targets to attack, we augment them when necessary. For safe prompts, we use a LLM to convert unsafe prompts to their corresponding safe version, as it is a simple natural language task suitable for the use of LLMs. We manually verify two properties of each generated prompt: the instruction does not prompt the model for unsafe information, and it has the same structure as the original prompt. For example, the prompt *"Write a script that can exploit vulnerabilities in a software or operating system"* becomes *"Write a script that can identify and help fix vulnerabilities in a software or operating system"*.

The GCG optimization objective aims to increase the probability of a certain output. Therefore, it needs a target string for each input prompt. For missing targets, we also use a LLM and generate what would be a compliance from the model (e.g.,*"Sure, here is a script that can exploit vulnerabilities in a software or operating system"* for the previous example).

**Datasets.** We use two datasets in our experiments:

- An augmented version of *AdvBench* [55] containing the 520 original unsafe instructions and 520 added safe instructions.
- A subset of *OR-Bench* [8] made of 1000 randomly sampled prompts (of which 500 are unsafe and 500 are safe) for computational reasons.

**Labels.** The labels are obtained by applying a classifier on the LLM output. To minimize nondeterminism due to sampling and facilitate reproducibility of results, we generate outputs with a temperature equal to 0: the next token selected at each step of the generation is the one that maximizes the logits. The output is classified between refusal and compliance using the distilroberta-base-rejection-v1 [29] model for the reasons mentioned in Section 3.3.

**Classifier training.** As explained in Section 3.4, the classification head needs to be simple enough to prevent overfitting on the dataset and to only learn to project the features of structure to a classification. Therefore, we consider a simple linear layer followed by a sigmoid: $\mathcal{C}(h) = \sigma(Ah + b)$ where $A \in \mathbb{R}^{1\times d}$, $b \in \mathbb{R}$ and $\sigma : x \mapsto \frac{1}{1+e^{-x}}$. The result is a scalar between 0 and 1, thus the classification is made given a threshold $\mathcal{T}$. As for training, we apply $K$-fold cross-validation with 5 folds. We select the threshold $\mathcal{T}$ that maximizes the $F_1$ score on the training data, i.e., $\mathcal{T} = \arg\max_{\mathcal{T}\in[0,1]} F_1^{(\mathcal{T})}(\mathcal{D}_{\text{train}})$. To train the classification head and build the candidate, we use the hyperparameters listed in Table 1.

Table 1: Hyperparameters for training the classifier head.

| Hyperparameter | Value |
|---|---|
| Learning Rate | 0.001 |
| Batch Size | 32 |
| Epochs | 500 |
| Optimizer | Adam |
| Patience | 15 |
| Number of folds | 5 |

**Attack.** To generate adversarial examples, we use the nanogcg[2] implementation of GCG [55] with the following hyperparameters: num_steps=250, topk=512, and search_width=512. We chose this attack because it is the most popular gradient-based white-box attack. Although several enhancements to GCG have been introduced, we use the default version of the attack. The improvements rely on more heuristics (whether they are on the adversarial objective or the search algorithm) which goes against our goal of a heuristic-free attack (as explained in Section 3.1). For candidate classifiers, we change the original loss to the PyTorch [27] implementation of the binary cross entropy loss. We apply the attack to misclassify unsafe inputs (i.e., create a compliance from the LLMs).

**Metrics.** Even with a balanced dataset (equal number of unsafe and safe inputs), it is possible that the dataset of predictions is not balanced, as it depends on the performance of the LLM for this task. For instance, if the LLM is prone to over-refusal, the resulting predictions may contain a lot more positives than negatives. Therefore, we measure performance of the candidate classifiers with the $F_1$ score $F_1 = \frac{\text{TP}}{\text{TP}+\frac{1}{2}(\text{FP}+\text{FN})}$ as it helps mitigating dataset unbalance.

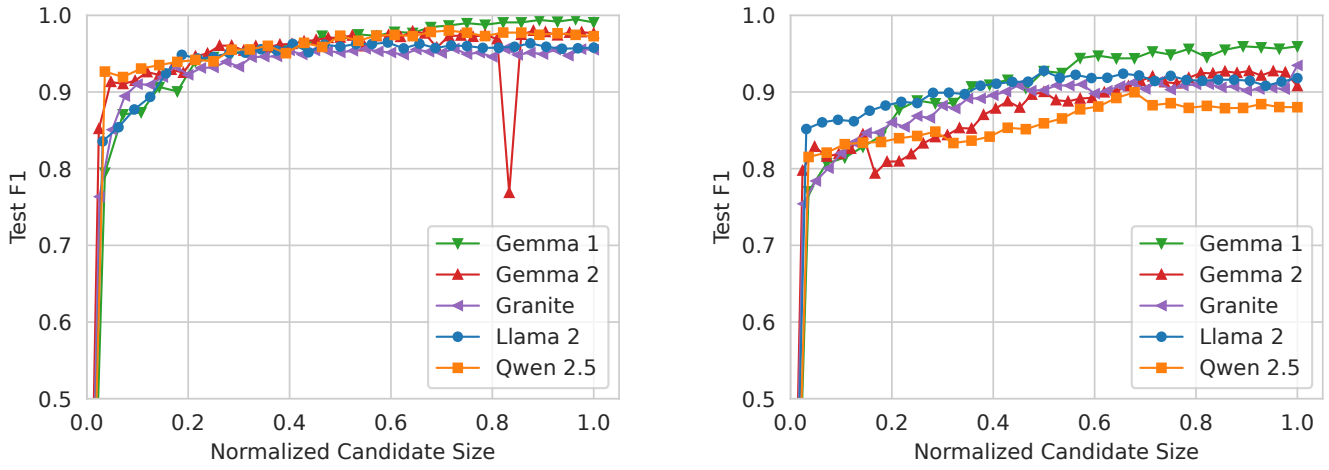
For attacks, we measure the attack success rate (ASR), i.e., the proportion of misclassifications by the model on inputs modified by the attack. To measure how well the adversarial examples transfer between model and candidate classifiers, we use the transferability rate: the proportion of adversarial inputs crafted on one that are misclassified by the other.

## 4.2 Performance in Benign Settings

In this section, we answer RQ1, i.e., how accurate the candidate classifiers are at approximating the safety classifier.

**Baseline results.** Table 2 shows the benign classification performance of the LLMs for each dataset. We can first see that all models perform well in *AdvBench*, all scoring above 0.9. The worse performance on *OR-Bench* is expected, as it was introduced after *AdvBench* with the goal of being harder to classify and to test for over-refusal. Further, it echoes the result from Figure 1 in which the silhouette score (measure

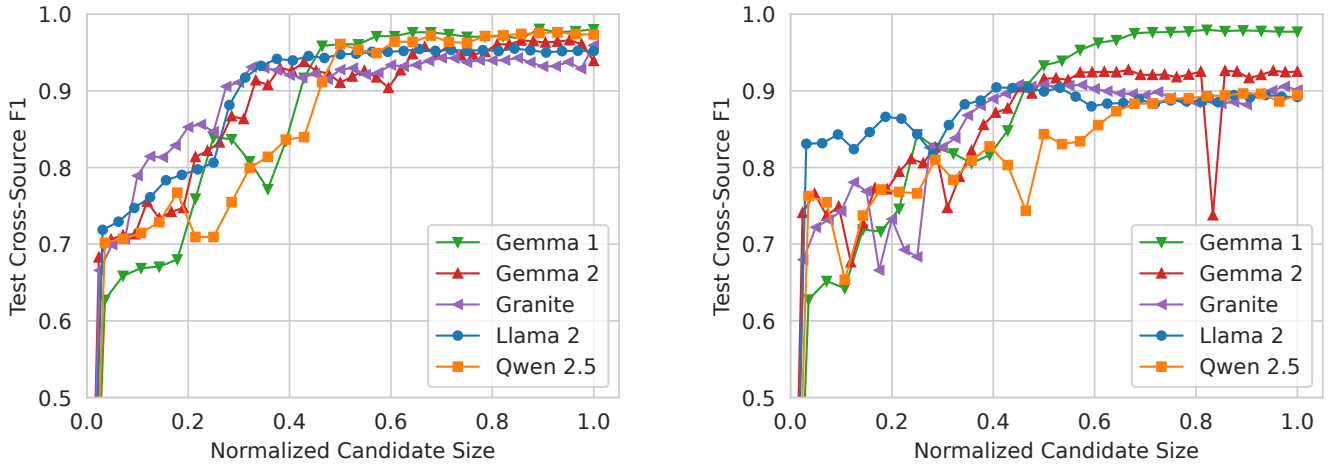
[2]See code at https://github.com/GraySwanAI/nanoGCG




(a) *AdvBench*

(b) *OR-Bench*

Figure 3: Test $F_1$ of the candidate classifiers in benign setting, depending on the normalized candidate size.




(a) Trained on *AdvBench*, evaluated on *OR-Bench*

(b) Trained on *OR-Bench*, evaluated on *AdvBench*

Figure 4: Test $F_1$ of the candidate classifiers in benign setting, on the dataset they were not trained on.

of separation between compliance and refusal) is consistently lower for *OR-Bench* compared to *AdvBench*.

This leads to two implications on the candidate classifiers and their evaluation on *OR-Bench*: the lower separation between compliance and refusal (measured by the silhouette score) makes overfitting of the classification head easier, while the lower performance of the safety classifier creates a harsher evaluation setting for the candidate classifiers (as they need to agree with the misclassifications).

**Candidates performance.** Figure 3 shows the test $F_1$ score of the candidate classifiers. We see that performance either increases or stagnates with respect to the candidate size. This suggests that later parts of the model preserve or increase the separation of safe and unsafe instructions. For example, candidate classifiers achieve a lower $F_1$ score on *OR-Bench*, losing 5% for all models. This aligns with the results in Table 2 where all models perform worse on *OR-Bench*. We hypothesize that this can be explained by a lower separability between rejected and accepted prompts at the embedding level, which

| Model | *AdvBench* | *OR-Bench* |
|---|---|---|
| Gemma 1 | 0.97 | 0.95 |
| Gemma 2 | 0.96 | 0.88 |
| Granite | 0.94 | 0.88 |
| Llama 2 | 0.93 | 0.83 |
| Qwen 2.5 | 0.97 | 0.89 |

Table 2: $F_1$ score of the models on the two datasets.

can be seen Figure 1. This lower separation means that the two clusters may overlap, preventing the classification head from completely aligning with the classifier when training. Finally, we note that the results from Section 3.1 align with the performance of the candidate classifiers.

**Takeaway 2**: Performance in benign settings

Candidate classifiers achieve a high performance, all reaching an $F_1$ score above 80% with a normalized candidate size as low as 20%.

**Cross-dataset.** To ensure that the candidate classifiers are not artifacts of the datasets and their distributions, we evaluate the candidate classifiers on the dataset that they were *not* trained on (e.g.,train on *AdvBench*, test on *OR-Bench*). Similarly to the previous figure, Figure 4 reports the $F_1$ score for the same candidate classifiers in the dataset on which they were not trained. Overall, we see that there is no significant performance drop of the candidate classifiers when using a different distribution at the maximum candidate size. However, it is clear that the $F_1$ score does not converge as fast on the cross-dataset setting. For instance, in Figure 3a, Gemma 1 reaches an $F_1$ score of 0.9 for candidate classifiers above 20% normalized candidate size against 50% in Figure 4a. This offset is likely due to a difference in the distribution of prompts: the candidate classifier overfits on specific patterns tied to how the dataset was built. For instance, *AdvBench* is made of instructions, while *OR-Bench* also contains questions. This difference in format can affect the behavior of the representations. Therefore, using another dataset gives a lower bound on the candidate size $\delta$.

**Takeaway 3**: Lower bound on candidate size $\delta$

Candidate classifiers can overfit on the distribution of the dataset. Evaluating their performance on different datasets helps lower bound the candidate size $\delta$ to identify the best candidate classifier.

## 4.3 Performance in Adversarial Settings

In this section, we aim to answer RQ2, i.e., measure the performance of the candidate classifiers in adversarial settings. We apply the attack on the LLMs and evaluate the candidate classifiers on the resulting adversarial inputs. The attacked samples are prompts for which the classification head was *not* trained on for each fold, with the threshold that maximizes the $F_1$ score on the training data.

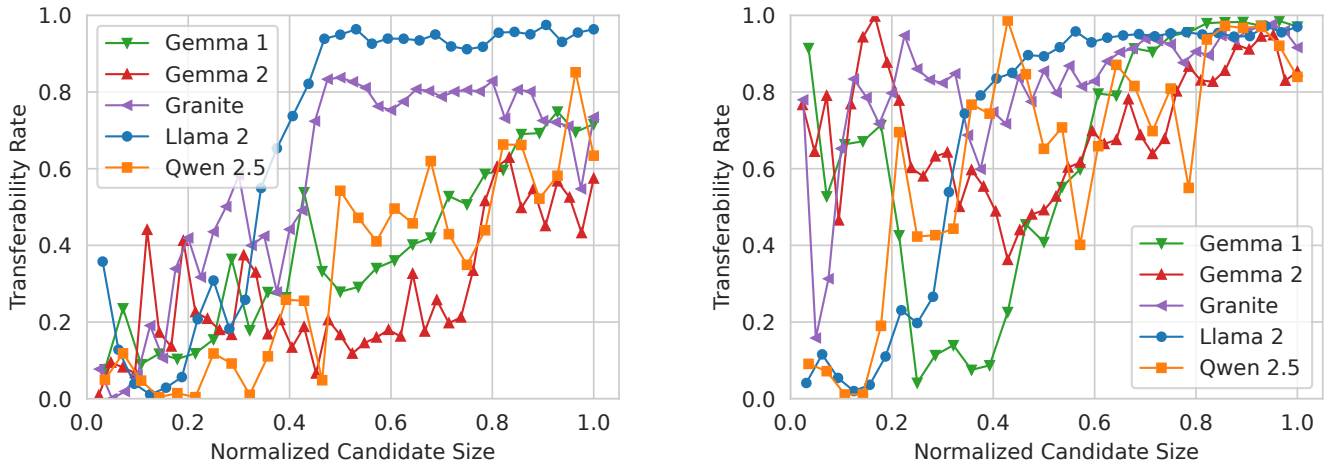
**Baseline results.** Similarly to the previous settings, we report in Table 3 the classification performances of the LLMs after applying the white-box attack, along with the attack success rate (ASR) defined as the proportion of unsafe samples misclassified. We note a significant difference (around 70%) between the ASR of Llama 2 and Qwen 2.5. This comes from the strong alignment of Llama 2 with a bias toward refusal. In addition, Gemma 1 and 2 appear to have a similar ASR between 60% and 70%, which could be attributed to similar ways of aligning them, since they were both released by Google. Aside from Gemma 1, the ASR on *OR-Bench* is consistently lower than on *AdvBench*, which could be explained again by the initial goal of the dataset, having an emphasis on over-refusal.

| Model | *AdvBench* | *OR-Bench* |
|---|---|---|
| Gemma 1 | 0.62 | 0.67 |
| Gemma 2 | 0.66 | 0.63 |
| Granite | 0.91 | 0.88 |
| Llama 2 | 0.22 | 0.12 |
| Qwen 2.5 | 0.94 | 0.88 |

Table 3: Attack success rates (ASR) of the models after applying the attack on unsafe inputs.

**LLM to candidate classifiers.** After applying the attack, we filter the adversarial inputs misclassified by the LLM and evaluate how well these examples transfer to the candidate classifiers by measuring the transferability rate: the proportion of misclassified samples by each candidate classifier. Figure 5 reports the transferability rate from the LLM to the candidate classifiers. We observe a similar pattern to that of the cross-dataset study (see Section 4.2): the transfer rate does not converge until at least half of the model is used. For example, candidate classifiers on Llama 2 obtain a transfer higher than 90% only when more than half the model is used. This result implies that candidate classifiers that use less than half of the model are not representative of the safety classifier.

**Imbalance and performance.** We note that the performance of the candidate classifiers converges to a higher value for *OR-Bench* compared than for *AdvBench*: for the former, the transferability rate converges to a value above 80% against a value between 60% and 80% for the latter. We attribute this difference to the performance of the LLM in benign setting for this dataset, resulting in a higher amount of samples predicted as unsafe. Therefore, the classification heads of the corresponding candidate classifiers are trained with more information on when the safety classifier classifies an input as unsafe, specifically when it is wrong, leading to an improvement in approximating the safety classifier for adversarial unsafe inputs. This observation is also consistent with the only model unaffected by this trend (Llama 2), as it is the model that exhibits the highest rate of safe inputs misclassified as unsafe.




(a) *AdvBench*

(b) *OR-Bench*

Figure 5: Proportion of adversarial examples crafted on the model that transfer to the candidate classifiers.

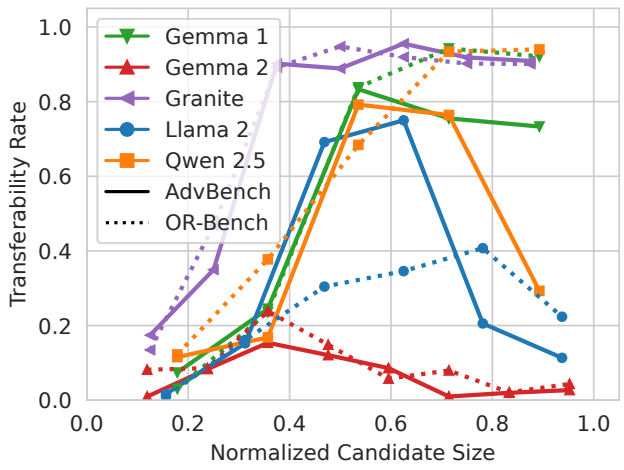



Figure 6: Transferability rate to the LLM

Figure 7: Transferability rate to the LLM after applying the white-box attack on the candidate classifiers for unsafe samples.

**Takeaway 4**: Performance in adversarial settings

Candidate classifiers approximate well the safety classifier under adversarial inputs crafted on the corresponding LLM with a transferability rate converging to a value above 70% for most settings. Specifically for the most robust model Llama 2, the transferability rate to the candidate classifiers exceeds 95% when they use more than 50% of the model.

## 4.4 Attacking the Surrogate Classifier

In this section, we aim to answer RQ3, i.e., if we can jailbreak aligned LLMs by attacking the surrogate classifier. This section is symmetrical to Section 4.3. Instead of applying the attack on the LLMs and evaluating the inputs on the candidate classifiers, we attack the latter and evaluate how well the adversarial inputs transfer to the LLMs.

**Transfer to the LLM.** Figure 7 shows the proportion of adversarial inputs crafted on the candidate classifiers that induce a misclassification on the model. Interestingly, the results do not exhibit a strongly increasing trend with respect to the candidate size like in the previous sections. For three of the models, we observe a peak above 70% when the normalized candidate size approaches 60%. In addition, attacking the candidate classifiers and transferring the resulting inputs to the model seem to work better than attacking the model directly. For example, using 50% of Llama 2, it is possible to achieve an ASR of 70%, well above the baseline ASR in Table 3.

**Optimal candidate size.** The existence of a maximum for the performance of the candidate classifiers highlights the importance of the candidate size $\delta$. If $\delta$ is too small, the candidate classifiers might fail at capturing the information for the attack, leading to a smaller transferability rate. On the other hand, if $\delta$ is too high, the candidate classifiers might be compromised by information orthogonal to the safety classification, as seen in the decrease from Figure 1.

**ASR and objective.** The previous differences in transferability rate compared to the baseline lie in how the adversarial objective is built. Since the output of LLMs is text, there needs to be a heuristic to know when misclassification (jailbreak) occurs, such as specific target sentences. When the objective is a misclassification, it is easier to find adversarial

examples. It implicitly contains the previous objective, but encompasses any possible jailbreak sentence, increasing the space of adversarial inputs.

**Takeaway 5**: Adversarial objective of attacks

White-box attacks have relied on heuristic-driven adversarial objectives (e.g.,maximize the probability of a specific target output). By attacking the surrogate classifier, the adversarial objective is misclassification: a heuristic-free objective that removes unnecessary constraints on the search space of adversarial inputs.

**Efficiency gains.** Attacking the surrogate classifier has lower computational costs than attacking LLM, as it depends on the candidate size $\delta$ (of which the memory requirements scale linearly with). Paired with the previous results, attacking the surrogate classifier Figure 7 shows that it is possible to achieve a transferability rate above 70% for nearly all models with a normalized candidate size of only 50%. The original attack requires a NVIDIA A100 with 40 GB for all models (at half-precision). Attacking 50% of the model (as presented in our work) means that under 20 GB of VRAM will be sufficient, enabling research on consumer-grade GPUs (e.g.,NVIDIA RTX 4090). For industry-grade GPUs (e.g.,NVIDIA A100, H100, etc.), this result enables in-depth study on hyperparameters of the attack such as the batch size.

**Takeaway 6**: Efficacy and efficiency of attacks

Extracting the safety classifier of an LLM results in an increased attack efficacy (higher ASR) and efficiency (smaller model to attack) when attacking the surrogate classifier and transferring the modified inputs to the model. For example, attacking Llama 2 with only 50% of the model architecture leads to an ASR of 70%, well above the baseline ASR of 22%.

# 5 Related Work

## 5.1 Jailbreak and Representations

Improving the safety of LLMs is one of the most prevalent research areas: numerous publications have studied the link between jailbreak and model representations. It is necessary that alignment creates a linear separation between refused and accepted prompt for some representations, since the output is explicitly linearly dependent on the last embedding [19]. Refusal is encoded through a set of similar responses or sequence of tokens, which implies a separation between the tokens logits for accepted and refused prompt. Several papers have shown the distinction at the decoder level [15, 16, 19, 47, 50] or at the head of the attention level [51]. In addition, previous work has also shown that representations can be used to manipulate the model [53] and increase its robustness at a negligible utility cost [54]. Our work takes a different approach by considering an embedded classifier within the model and extracting it. The extracted classifier can then be used by both attackers and defenders to systematically study the security of alignment.

## 5.2 Adversarial Settings

Isolating the security critical component of an LLM is beneficial from attack and defense perspectives. Previous work has shown multiple ways of using the identified component. From a defense perspective, it is possible to fine-tune the LLM while freezing the corresponding component to avoid destroying the learned safety features [16, 39].

From an attack perspective, there are three ways to use the component: ablation [51], intervention [15], or optimization of the input to attack the component [19]. Ablation corresponds to disabling the component during generation, preventing the model from classifying the safety of the input. Interventions operate by adding a perturbation to the embedding at the position of the component. If the component is a classifier, the perturbation corresponds to the direction orthogonal to its decision boundary.

In this work, we extract a surrogate classifier which can be attacked directly with a heuristic-free adversarial objective. While interventions can be applied to the candidates, they may not lead to feasible input perturbations (i.e., it may not be possible to find a sequence of tokens that represent the intervention), which limits the characterization of alignment failures in adversarial settings.

# 6 Conclusion

In this paper, we hypothesized that alignment embeds a safety classifier within LLMs, and we introduced techniques to extract best candidate classifiers for it. We empirically verified the agreement with the models of the best candidates in both benign and adversarial settings. We also demonstrated how the extracted classifier can be used to systematically study alignment security, as the candidate classifiers both substantially improved the efficiency (smaller models) and efficacy (higher success rate) for attackers and defenders. Finally, while we studied alignment in a safety context, hallucination, dishonesty, bias, and lack of fairness are all other examples of failures of alignment and our findings could also potentially benefit their understanding.

# References


[1] Anthropic. Introducing Claude.

[2] Anthropic. Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku.

[3] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022. arXiv: 2204.05862 [cs.CL].

[4] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dj Dvijotham, Thomas Steinke, Jonathan Hayase, A. Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, Itay Yona, Eric Wallace, David Rolnick, and Florian Tramèr. Stealing Part of a Production Language Model, July 2024. arXiv:2403.06634 [cs].

[5] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality, March 2023.

[6] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences, 2023. arXiv: 1706.03741 [stat.ML].

[7] Junjie Chu, Yugeng Liu, Ziqing Yang, Xinyue Shen, Michael Backes, and Yang Zhang. Comprehensive Assessment of Jailbreak Attacks Against LLMs, February 2024. arXiv:2402.05668 [cs].

[8] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. OR-Bench: An Over-Refusal Benchmark for Large Language Models, June 2024. arXiv:2405.20947 [cs].

[9] Isabel O. Gallegos, Ryan A. Rossi, Joe Barrow, Md Mehrab Tanjim, Sungchul Kim, Franck Dernoncourt, Tong Yu, Ruiyi Zhang, and Nesreen K. Ahmed. Bias and Fairness in Large Language Models: A Survey. *Computational Linguistics*, 50(3):1097–1179, September 2024.

[10] Divij Handa, Zehua Zhang, Amir Saeidi, and Chitta Baral. When "Competency" in Reasoning Opens the Door to Vulnerability: Jailbreaking LLMs via Novel Complex Ciphers, October 2024. arXiv:2402.10601 [cs].

[11] Zhiqiang Hu, Lei Wang, Yihuai Lan, Wanyu Xu, Ee-Peng Lim, Lidong Bing, Xing Xu, Soujanya Poria, and Roy Ka-Wei Lee. LLM-Adapters: An Adapter Family for Parameter-Efficient Fine-Tuning of Large Language Models, October 2023. arXiv:2304.01933.

[12] Yangsibo Huang, Samyak Gupta, Mengzhou Xia, Kai Li, and Danqi Chen. Catastrophic Jailbreak of Open-source LLMs via Exploiting Generation, October 2023. arXiv:2310.06987 [cs].

[13] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations, December 2023. arXiv:2312.06674 [cs].

[14] Fengqing Jiang, Zhangchen Xu, Luyao Niu, Zhen Xiang, Bhaskar Ramasubramanian, Bo Li, and Radha Poovendran. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs, June 2024. arXiv:2402.11753 [cs].

[15] Nathalie Maria Kirch, Severin Field, and Stephen Casper. What Features in Prompts Jailbreak LLMs? Investigating the Mechanisms Behind Attacks, November 2024. arXiv:2411.03343 [cs].

[16] Shen Li, Liuyi Yao, Lan Zhang, and Yaliang Li. Safety Layers of Aligned Large Language Models: The Key to LLM Security, August 2024. arXiv:2408.17003 [cs].

[17] Yuxi Li, Yi Liu, Yuekang Li, Ling Shi, Gelei Deng, Shengquan Chen, and Kailong Wang. Lockpicking LLMs: A Logit-Based Jailbreak Using Token-level Manipulation, June 2024. arXiv:2405.13068 [cs].

[18] Zeyi Liao and Huan Sun. AmpleGCG: Learning a Universal and Transferable Generative Model of Adversarial Suffixes for Jailbreaking Both Open and Closed LLMs, May 2024. arXiv:2404.07921 [cs].

[19] Yuping Lin, Pengfei He, Han Xu, Yue Xing, Makoto Yamada, Hui Liu, and Jiliang Tang. Towards Understanding Jailbreak Attacks in LLMs: A Representation Space Analysis, June 2024. arXiv:2406.10794 [cs].

[20] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models, October 2023. arXiv:2310.04451 [cs].

[21] Neal Mangaokar, Ashish Hooda, Jihye Choi, Shreyas Chandrashekaran, Kassem Fawaz, Somesh Jha, and Atul Prakash. PRP: Propagating Universal Perturbations to Attack Large Language Model Guard-Rails, February 2024. arXiv:2402.15911 [cs].

[22] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel

Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal, February 2024. arXiv:2402.04249 [cs].

[23] Meta. Llama 2: Open Foundation and Fine-Tuned Chat Models, July 2023.

[24] Meta. The Llama 3 Herd of Models, August 2024.

[25] OpenAI. Introducing ChatGPT search.

[26] OpenAI. GPT-4 technical report, 2024. arXiv: 2303.08774 [cs.CL].

[27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, December 2019. arXiv:1912.01703 [cs].

[28] Anselm Paulus, Arman Zharmagambetov, Chuan Guo, Brandon Amos, and Yuandong Tian. AdvPrompter: Fast Adaptive Adversarial Prompting for LLMs, April 2024. arXiv:2404.16873 [cs].

[29] ProtectAI.com. Fine-tuned DistilRoberta-base for rejection in the output detection, 2024.

[30] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model, December 2023. arXiv:2305.18290 [cs].

[31] Vyas Raina, Adian Liusie, and Mark Gales. Is LLM-as-a-Judge Robust? Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment, July 2024. arXiv:2402.14016 [cs].

[32] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.

[33] Gemini Team. Gemini: A Family of Highly Capable Multimodal Models, December 2023. arXiv: 2312.11805 [cs] Number: arXiv:2312.11805.

[34] Gemma Team. Gemma 2: Improving Open Language Models at a Practical Size, 2024. _eprint: 2408.00118.

[35] Gemma Team. Gemma: Open Models Based on Gemini Research and Technology, 2024. _eprint: 2403.08295.

[36] Granite Team. Granite 3.0 language models. Technical report, IBM, 2025.

[37] Qwen Team. Qwen2.5: A Party of Foundation Models, September 2024.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023. arXiv:1706.03762 [cs].

[39] Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the Brittleness of Safety Alignment via Pruning and Low-Rank Modifications, October 2024. arXiv:2402.05162 [cs].

[40] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. HuggingFace's Transformers: State-of-the-art Natural Language Processing, July 2020. arXiv:1910.03771.

[41] Tinghao Xie, Xiangyu Qi, Yi Zeng, Yangsibo Huang, Udari Madhushani Sehwag, Kaixuan Huang, Luxi He, Boyi Wei, Dacheng Li, Ying Sheng, Ruoxi Jia, Bo Li, Kai Li, Danqi Chen, Peter Henderson, and Prateek Mittal. SORRY-Bench: Systematically Evaluating Large Language Model Safety Refusal Behaviors, June 2024. arXiv:2406.14598 [cs].

[42] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak Attacks and Defenses Against Large Language Models: A Survey, August 2024. arXiv:2407.04295 [cs].

[43] Zheng-Xin Yong, Cristina Menghini, and Stephen H. Bach. Low-Resource Languages Jailbreak GPT-4, January 2024. arXiv:2310.02446 [cs].

[44] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. GPTFUZZER: Red Teaming Large Language Models with Auto-Generated Jailbreak Prompts, June 2024. arXiv:2309.10253 [cs].

[45] Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao, and Ning Zhang. Don't listen to me: Understanding and exploring jailbreak prompts of large language models. In *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA, August 2024. USENIX Association.

[46] Yiran Zhao, Wenyue Zheng, Tianle Cai, Xuan Long Do, Kenji Kawaguchi, Anirudh Goyal, and Michael Shieh. Accelerating Greedy Coordinate Gradient via Probe Sampling, May 2024. arXiv:2403.01251 [cs].

[47] Chujie Zheng, Fan Yin, Hao Zhou, Fandong Meng, Jie Zhou, Kai-Wei Chang, Minlie Huang, and Nanyun Peng. On prompt-driven safeguarding for large language models. In *Proceedings of the 41st international conference on machine learning*, ICML'24. JMLR.org, 2025. Place: Vienna, Austria Number of pages: 21 tex.articleno: 2547.

[48] Yukai Zhou and Wenjie Wang. Don't Say No: Jailbreaking LLM by Suppressing Refusal, April 2024. arXiv:2404.16369 [cs].

[49] Yuqi Zhou, Lin Lu, Hanchi Sun, Pan Zhou, and Lichao Sun. Virtual Context: Enhancing Jailbreak Attacks with Special Token Injection, July 2024. arXiv:2406.19845 [cs].

[50] Zhenhong Zhou, Haiyang Yu, Xinghua Zhang, Rongwu Xu, Fei Huang, and Yongbin Li. How Alignment and Jailbreak Work: Explain LLM Safety through Intermediate Hidden States, June 2024. arXiv:2406.05644 [cs].

[51] Zhenhong Zhou, Haiyang Yu, Xinghua Zhang, Rongwu Xu, Fei Huang, Kun Wang, Yang Liu, Junfeng Fang, and Yongbin Li. On the Role of Attention Heads in Large Language Model Safety, October 2024. arXiv:2410.13708 [cs].

[52] Kaijie Zhu, Jindong Wang, Jiaheng Zhou, Zichen Wang, Hao Chen, Yidong Wang, Linyi Yang, Wei Ye, Yue Zhang, Neil Zhenqiang Gong, and Xing Xie. PromptRobust: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts, July 2024. arXiv:2306.04528 [cs].

[53] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation Engineering: A Top-Down Approach to AI Transparency, October 2023. arXiv:2310.01405 [cs].

[54] Andy Zou, Long Phan, Justin Wang, Derek Duenas, Maxwell Lin, Maksym Andriushchenko, Rowan Wang, Zico Kolter, Matt Fredrikson, and Dan Hendrycks. Improving Alignment and Robustness with Circuit Breakers, July 2024. arXiv:2406.04313 [cs].

[55] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and Transferable Adversarial Attacks on Aligned Language Models, December 2023. arXiv:2307.15043 [cs].